



BOOTCAMP FRONTEND & REACT · SEMANA 2

# Tailwind

Clase 6 — Tailwind CSS v4: la misma tienda, escrita con utilidades, y los componentes reutilizables

Clase virtual · L·M·V

Proyecto TechCart

STEER  
RC  
LA

# Agenda

## 1. Qué es Tailwind

Qué problema resuelve, cuál no, y la crítica honesta

## 3. Las utilidades

Espacio, letra, color, tamaño, borde, sombra, flex y grid

## 5. Re-skin: cabecera y menú

Regla por regla, comparando con lo que escribimos

## 7. Modo oscuro y animaciones

dark: · animate-\* con los @keyframes en CSS · motion-reduce:

## 2. Cómo se pone a andar

El CDN de navegador de la v4 · Preflight

## 4. Tokens con @theme

El :root de la Clase 5, mudado: cada token genera utilidades

## 6. Variantes

Responsive mobile first · hover, focus-visible, active, disabled

## 8. Componentes reutilizables

Las tres vías · @apply dentro de @layer components

 **Receso de 15 min** después de las variantes · cierre: qué se queda en CSS, errores frecuentes, resumen, 5 tareas y anticipo

# Dos de memoria

¿Qué significa que una hoja esté escrita `mobile first`?

¿Qué hace en nuestra hoja `@media (prefers-color-scheme: dark)`?

LA MISMA TARJETA, EL MISMO DIBUJO EN PANTALLA · LO QUE CAMBIA ES DÓNDE VIVE EL CSS

# Una utilidad = **una propiedad**

Como lo escribimos nosotros

```
<article class="tarjeta">...</article>

/* ...y en otro archivo: */
.tarjeta {
  display: flex;
  flex-direction: column;
  background-color: #fff;
  border: 1px solid #d2d2d7;
  border-radius: 12px;
  padding: 1rem;
}
```

Como lo escribe Tailwind

```
<article class="flex flex-col bg-white
  border border-#[d2d2d7] rounded-xl p-4">
  ...
</article>

/* ...y ningún archivo debajo */
```

**flex** = display:flex · **flex-col** = flex-direction:column · **p-4**  
= padding:1rem · **rounded-xl** = border-radius:12px

ANTES DE APRENDERLO, SABER PARA QUÉ SIRVE · Y PARA QUÉ NO

# Qué resuelve y qué no

## Lo que **sí** resuelve

- Ponerle **nombre** a cada cosa: deja de ser una decisión
- La hoja que **solo crece** y que nadie se anima a borrar
- El **miedo a tocar**: lo que cambio afecta a un elemento y no a otros
- Los **valores inventados**: hay una escala, y se respeta

## Lo que **no** resuelve

- No enseña a maquetar: si no sabes Flexbox, no sabes qué pedir
- No arregla un HTML mal armado
- No reemplaza saber CSS: son las mismas propiedades
- No es más rápido que un CSS bien escrito

**La crítica, y es cierta:** "el HTML se llena de clases". La respuesta tiene dos partes: esa información antes estaba repartida en dos archivos (se mudó, no apareció), y cuando una ristra se repite muchas veces, eso es una señal. Qué hacer con ella es el segundo tema de hoy.

EL MISMO TAILWIND · LO QUE CAMBIA ES CUÁNDO SE GENERA EL CSS

## Dos caminos: **npm** o el CDN de navegador

### En un proyecto real · **npm**

```
npm install tailwindcss
```

- El CSS se genera **antes de publicar**
- Queda un **.css normal**, con solo las utilidades usadas
- Necesita **Node** · llega en la semana 5, con React

### Hoy en clase · **CDN de navegador**

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
```

- Tailwind trabaja **dentro del navegador**, en vivo
- Sin instalar nada · perfecto para aprender y prototipar
- Más lento al cargar: **no es para producción**

Si un tutorial dice **cdn.tailwindcss.com**, es de la versión 3. El de la v4 es el paquete **@tailwindcss/browser**.

EL RESET QUE VIENE INCLUIDO · NADA APARECE SI NO LO PIDES

# Lo que **apaga** y lo que regala

## Apaga

- Márgenes y padding de todo: **en cero**
- Los títulos pierden **tamaño y negrita**: un h1 se ve como un párrafo
- Las listas pierden **viñetas y sangría**
- El `<address>` pierde la cursiva

## Regala **tres reglas nuestras**

- **box-sizing: border-box** para todo (Clase 2)
- **img { max-width:100%; height:auto }** (Clase 5)
- **input, select, textarea, button { font: inherit }** (Clase 2)

Por eso al probar Tailwind por primera vez parece que "se rompió todo". No se rompió: se apagó lo que el navegador ponía gratis. En un proyecto que ya existe, hay que revisar el HTML entero. Eso es exactamente lo que hacemos hoy.

LA FAMILIA QUE MÁS SE USA · EL NÚMERO SE MULTIPLICA POR 4 PÍXELES

## La escala va de 4 en 4

|      |                     |
|------|---------------------|
| p-1  | 4px                 |
| p-2  | 8px                 |
| p-3  | 12px                |
| p-4  | 16px ← nuestro 1rem |
| p-6  | 24px                |
| p-8  | 32px                |
| p-12 | 48px                |

```
/* el valor base es una variable, en rem */
--spacing: 0.25rem; /* = 4px */

.p-4 { padding: calc(var(--spacing) * 4) }
```

**p-4 · px-5 · py-4**

los cuatro lados · izq+der ·  
arriba+abajo

**pt pr pb pl**

un solo lado

**m-4 · mx-auto · mt-auto**

margen, con los mismos ejes

**gap-4 · gap-x-4 · gap-y-6**

el hueco de flex y de grid

La escala está en **rem**: si la persona agranda la letra de su navegador, los espacios crecen con ella. Es la decisión de accesibilidad de la Clase 5, ya tomada.

LETRA, COLOR, TAMAÑO, BORDE Y SOMBRA · EL MISMO PATRÓN EN TODAS

# Las familias de utilidades

## Letra

`text-sm` · `text-lg` ·  
`text-3xl`

font-size: 14 · 18 · 30px

`font-semibold` ·  
`font-bold`

font-weight: 600 · 700

`leading-tight` ·  
`tracking-wide`

line-height · letter-spacing

`uppercase` · `italic`  
· `text-center`

la propiedad completa

## Color, tamaño, borde y sombra

`bg-*` · `text-*` ·  
`border-*`

fondo · letra · borde

`w-35` · `h-11` · `w-`  
`full`

140px · 44px · 100%

`max-w-*` · `min-h-*` ·  
`aspect-square`

topes y proporción

`border` · `border-2` ·  
`rounded-xl`

1px · 2px · radio de 12px

`rounded-lg` ·  
`rounded-full`

8px · la píldora

`shadow-sm` ... `shadow-`  
`2xl`

cinco sombras ya armadas

`object-contain` ·  
`overflow-x-auto`

las mismas de la Clase 5

`w-[137px]`

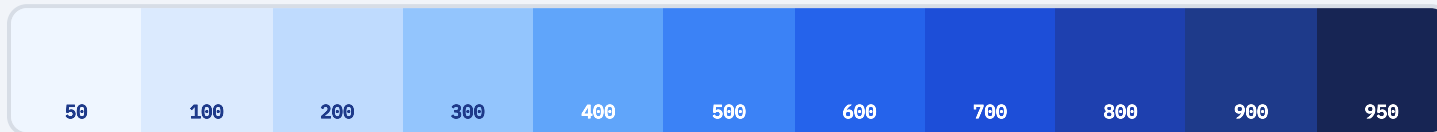
valor arbitrario: solo si no hay  
escalón

Ojo con **text-**, que hace dos trabajos: si lo que sigue es un tamaño es **font-size**; si es un color, es **color**.

LOS COLORES DE FÁBRICA VIENEN EN ESCALAS DE ONCE ESCALONES

De **50** a **950**

bg-blue-\*



**50** es casi blanco · **500** es el color pleno · **950** es casi negro. Cada familia (slate, red, green, blue...) trae los once.

En modo oscuro se sube dos o tres escalones para que el mismo color siga contrastando. Es exactamente lo que hicimos a ojo la clase pasada al aclarar el azul de la marca: la escala existe para no hacerlo a ojo.

AQUÍ NO HAY NADA NUEVO: SON LAS CLASES 3 Y 4 CON NOMBRE CORTO

## Lo que ya saben, traducido

`display: flex` → **flex**

`display: grid` → **grid**

`flex-direction: column` → **flex-col**

`grid-template-columns: repeat(3, 1fr)` → **grid-cols-3**

`justify-content: space-between` → **justify-between**

`grid-column: span 2` → **col-span-2**

`align-items: center` → **items-center**

`grid-column: 1 / -1` → **col-span-full**

`flex-wrap: wrap` → **flex-wrap**

`grid-auto-rows: 1fr` → **auto-rows-fr**

`gap: 16px` → **gap-4**

`align-self: center` → **self-center**

`margin-top: auto` → **mt-auto**

`order: -1` → **order-first**

No se aprenden de memoria: se busca la **propiedad de CSS** en [tailwindcss.com/docs](https://tailwindcss.com/docs) y la página muestra la utilidad y el CSS exacto, uno al lado del otro.

## El **:root** de la Clase 5, mudado

Clase 5 · `css/styles.css`

```
:root {
  --marca: #0066cc;
}
```

Clase 6 · `<style type="text/tailwindcss">`

```
@theme {
  --color-marca: #0066cc;
}

/*y aparecen solas, sin escribirlas: */
bg-marca  text-marca  border-marca
```

El **prefijo** del nombre dice de qué familia es. No es decoración: es la instrucción.

**--color-\***

bg- · text- · border-

**--radius-\***

rounded-

**--text-\***

tamaños de letra

**--font-\***

tipografías

**--animate-\***

animate-

**--breakpoint-\***

prefijos responsive propios

**--container-\***

max-w-

**--spacing**

la escala entera de 4 en 4

En **v3** esto se configuraba en un archivo aparte, **tailwind.config.js**. En **v4** vive en el CSS. Si un tutorial pide ese archivo, es de la versión anterior.

TAILWIND ES MOBILE FIRST DE FÁBRICA · TODOS LOS PREFIJOS SON MIN-WIDTH

# La utilidad **sin prefijo** es la base

**base**  
celular

**sm:**  
640px

**md:**  
768px = 48em

**lg:**  
1024px = 64em

**xl: · 2xl:**  
1280 · 1536px

```
<!-- Clase 5, a mano -->
header { flex-direction: column }
@media (min-width: 48em) {
  header { flex-direction: row }
}

<!-- Clase 6, con variantes -->
class="flex-col md:flex-row"
```

## Lo que hay que leer

- **md:** es "de 768 para arriba", nunca "hasta 768"
- md: y lg: son **exactamente** nuestros 48em y 64em
- Las variantes **se apilan**: md:hover:bg-marca
- El orden en que las escribes **no importa**: Tailwind las ordena
- Si tu corte no está en la escala, se **declara** con --breakpoint-\*

## LAS MISMAS PSEUDO-CLASES DE LAS CLASES 2 Y 5, COMO PREFIJO

Variantes de **estado****hover:** = `:hover` · el mouse encima**focus-visible:** = `:focus-visible` · el teclado**active:** = `:active` · mientras se aprieta**dark:** = `@media (prefers-color-scheme: dark)`**motion-reduce:** = `@media (prefers-reduced-motion: reduce)`**disabled:** = `:disabled` · el control apagado**user-invalid:** = `:user-invalid` · inválido tras tocarlo**even:** · **last:** = `:nth-child(even)` · `:last-child`**before:** · **after:** = `::before` · `::after`**placeholder:** = `::placeholder`**group** · el padre manda sobre el hijo

```
<article class="group">
  <h3 class="group-hover:text-marca">
```

A mano sería **.tarjeta:hover h3**: una regla con dos partes. Con utilidades sueltas no se puede decir; por eso existe.

**peer** · un hermano manda sobre otro

```
<input class="peer">
<label class="peer-user-invalid:text-error">
```

La etiqueta se pone roja cuando el campo de al lado es inválido. Queda anotado; hoy no lo usamos.

# Modo oscuro con **dark:** y con tokens

## 1 · La variante **dark:**, elemento por elemento

```
class="bg-white dark:bg-slate-900
      text-slate-900 dark:text-slate-50"
```

Se lee sola y es perfecta para una pieza suelta. Si la página entera cambia de piel, hay que escribirla dos veces en 40 elementos.

## 2 · Los **tokens**, una sola vez

```
@media (prefers-color-scheme: dark) {
  :root { --color-fondo: #111114; ... }
}
```

Ninguna utilidad cambia: **bg-tarjeta** por dentro es **var(--color-tarjeta)**. Es el bloque de la Clase 5, tal cual.



De fábrica **dark:** sigue al **sistema operativo**: no hay ningún botón que lo active. Para el interruptor de sol y luna hay que cambiar la estrategia a "por clase" y mover un atributo con JavaScript.

UN REPARTO EN TRES PARTES · Y UN BUEN EJEMPLO DE QUÉ HACE TAILWIND Y QUÉ NO

## Los @keyframes se escriben a mano

### 1 · css/styles.css

```
@keyframes aparecer {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
```

### 2 · @theme

```
--animate-aparecer:
  aparecer 0.5s ease-out
  backwards;
```

Es el mismo valor que  
escribíamos en la propiedad  
**animation**.

### 3 · en el elemento

```
class="animate-aparecer
  motion-reduce:animate-
  none"
```

**backwards** y no **both**: con both el  
último fotograma se queda  
pegado y el hover deja de  
moverse.

No hay ni va a haber una utilidad que **describa** fotogramas: un @keyframes es una descripción libre, no una propiedad con un valor. Lo que sí existe es el token que le pone nombre.

**motion-reduce**: va en cada elemento que anime: se ve al leer el HTML, pero hay que acordarse de escribirlo. La media query al final del archivo los agarraba a todos juntos. Las dos formas son correctas.

# Tres vías, y **cuándo** cada una

## 1 · Repetir las utilidades

- Es **lo normal** y lo que más se usa
- Si aparece dos o tres veces, **no toques nada**
- Extraer cuesta: pierdes leer el elemento de un vistazo

## 2 · Una clase propia con @apply

- Se repite **cinco veces o más**
- Es **siempre igual** entre un uso y otro
- Tiene un **nombre obvio** para todo el equipo
- Las tres a la vez, o no lo extraigas

## 3 · Un componente de React

- Empaqueta las clases **y el contenido y el comportamiento**
- Una función Tarjeta que recibe nombre, precio y foto
- Es el destino de verdad · **semana 5**

La pregunta que decide es siempre la misma: **¿cuántas veces se repite, y siempre igual?** En TechCart: la tarjeta 7 veces, el campo 8, el botón 5, el título 3. La píldora "Destacado" lleva 12 clases y aparece **una sola vez**: no se extrae.

PEGA ADENTRO DE UNA CLASE PROPIA LO QUE HARÍAN ESAS UTILIDADES

## @apply dentro de @layer components

```
@layer components {
  .tarjeta {
    @apply bg-tarjeta border border-borde
      rounded-tarjeta p-4;
  }
}
```

### Por qué dentro de la **capa**

- Las **capas** se ordenan entre ellas y ganan las de abajo, **sin mirar la especificidad**
- Las utilidades van **después** de los componentes
- Así la destacada puede pedir **bg-resalte border-2** sobre **.tarjeta** y ganar
- Sin capa, la clase ganaría siempre y no habría excepciones

**@apply solo funciona en CSS que pase POR Tailwind:** el bloque `<style type="text/tailwindcss">`, o el archivo de entrada si el proyecto usa npm. Escrito en **css/styles.css** no hace absolutamente nada, y **no da ningún error:** el navegador ve una regla que no entiende y la descarta en silencio.

Lo que entra en el componente es solo lo que es igual en **todos** los usos. El flex, la animación, el hover y el bloque de 2x2 se quedan afuera, como utilidades en cada elemento. Si una declaración no vale para todos, no entra.

USAR TAILWIND NO SIGNIFICA NO ESCRIBIR CSS NUNCA MÁS · DE 82 REGLAS QUEDARON 9

## Lo que **se queda** en CSS

### Tres cosas, y el **porqué**

- **El plano con `grid-template-areas`.** Escrito como utilidad sería un valor arbitrario de tres líneas entre corchetes, ilegible. Y con el plano se quedan sus puntos de corte.
- **Los `@keyframes`.** No hay utilidad que describa fotogramas.
- **`color-scheme: light dark`.** No es un estilo: es un aviso al navegador sobre los controles que dibuja él.

Quién le gana a quién · las capas de la cascada

`@layer theme` – los tokens

`@layer base` – Preflight

`@layer components` – `.tarjeta`, `.boton...`

`@layer utilities` – `flex`, `p-4`, `md:flex-row`

**tu CSS SIN capa** – le gana a todo

Hasta hoy sabíamos dos reglas: gana el más específico, y a igual especificidad gana el de más abajo. Hoy se suma una tercera, **por encima de las dos**: el CSS que no está en ninguna capa le gana a todo el que sí lo está. Y **todo Tailwind está en capas**. Por eso nuestra hoja quedó corta y toca solo propiedades que ninguna utilidad usa.

LOS TROPIEZOS DEL TEMA · NO SE CODEAN, SE RECONOCEN

## Errores frecuentes

- 1 Clases inventadas.** `bg-azul-marca`, `text-grande`: no existen, no generan CSS y **no dan error**. Si no hace nada, primero revisa que exista.
- 2 Valores arbitrarios abusados.** `w-[137px]`, `p-[13px]`: si todo va entre corchetes, tiraste la escala y volviste a los valores sin criterio.
- 3 Invertir los cortes.** `md:flex-col` no es "en el celular en columna": es de 768 para arriba. La utilidad sin prefijo es la base, y la base es el celular.
- 4 Mezclar la hoja a mano con las utilidades.** Gana la hoja a mano, porque está fuera de las capas. No da ningún error: es el que más tiempo hace perder.
- 5 Esperar que `dark` funcione con un botón.** De fábrica sigue al sistema operativo. Para el botón hay que cambiar la estrategia a "por clase".
- 6 `@apply` en un archivo que Tailwind no lee.** En `css/styles.css` no pasa nada y no hay error. Va en el bloque `type="text/tailwindcss"`.
- 7 `transition` a secas.** Anima una lista larga que incluye todos los colores, así que al cambiar de modo la página hace un fundido raro. Nombra las propiedades.
- 8 Olvidar que Preflight apagó cosas.** "Se fueron los puntos de la lista", "el título se ve chiquito". No se rompió: hay que pedirlo (`list-decimal`, `text-lg`).

LO QUE VIMOS HOY · TAILWIND NO REEMPLAZA TU CSS, ES OTRA FORMA DE ESCRIBIRLO

# Resumen

## utilidad

una clase, una propiedad. El CSS se muda al HTML.

## @tailwindcss/browser@4

el CDN de hoy. En proyecto real, npm: llega con React.

## Preflight

el reset incluido. Regala tres reglas nuestras.

## --spacing · de 4 en 4

p-4 = 16px. En rem, así que acompaña a quien agranda la letra.

## text- bg- border- w- h- rounded- shadow-

las familias, con el mismo patrón.

## flex · grid · gap-4 · col-span-full

las Clases 3 y 4 con nombre corto.

## md: lg: xl:

todos min-width: mobile first de fábrica.

## hover: focus-visible: active: disabled:

las pseudo-clases, como prefijo. Y group / peer.

## @theme

el :root de la Clase 5. Cada token genera sus utilidades.

## dark:

para las excepciones. La paleta, por tokens.

## animate-\* + @keyframes

el nombre como token, los fotogramas a mano.

## @apply en @layer components

solo si se repite mucho, siempre igual y tiene nombre obvio.

## 5 tareas

### FÁCIL

**Replicar la clase** (CDN, @theme, modo oscuro, utilidades por zona y los cuatro componentes) y agregar un **cuarto beneficio** usando `.tarjeta`. Comprobar a 390, 768 y 1400 que auto-fit lo acomoda solo.

### INTERMEDIA 1

**Un token nuevo, de punta a punta.** `--color-oferta` en @theme y en el bloque oscuro, y una píldora "Oferta" en la tarjeta del iPad con `before:.` Token → utilidad → modo oscuro, sin tocar ninguna regla.

### INTERMEDIA 2

**Un quinto componente.** Las 12 celdas de la tabla repiten las mismas 4 clases. Decidan si merece una clase `.celda` con `@apply`; si sí, que los encabezados sigan con su fondo distinto. Si no, escríbanme por qué. **Las dos respuestas valen** si el argumento está.

### DIFÍCIL 1

**Entrada escalonada.** Busquen "delay" en la documentación y denle 100, 200 y 300 ms a la segunda, tercera y cuarta tarjeta. Ahí se entiende el `backwards`. Comprobar que con movimiento reducido también se apaga.

### DIFÍCIL 2

**Interruptor de modo oscuro, sin JavaScript todavía.** Lean "Dark mode" y cambien la estrategia para que `dark:` mire un atributo del `<html>` en vez del sistema. Demostrarlo cambiando el atributo a mano en DevTools.

Próxima clase

# De verse bien... a responder

La tienda se ve bien, se adapta, cambia de piel de noche y está escrita como se escribe en la industria. Pero todavía **no hace nada**: el botón "Agregar al carrito" no agrega nada a ningún carrito. La próxima clase empieza **JavaScript**: variables, funciones, eventos y el **DOM**, que es la forma en que JavaScript ve el HTML.

**Pista para casa:** entra a [play.tailwindcss.com](https://play.tailwindcss.com) y prueba ahí una utilidad que hoy solo nombramos, como `peer` o `group-focus`. No vas a romper nada.